

# MahaVistaar, *in production*: the serving stack, in numbers.

*A fine-tuned 27B-parameter model, four H100s, and a 100-concurrent-call threshold. What it costs, what it can hold, and the design choices behind a dual-provider inference stack that puts latency ahead of utilisation.*

---

## AT A GLANCE

~80 concurrent users

per 4-GPU MahaVistaar node under ideal conditions; 60–70 the safe operational range.

₹0.05 / question

on the self-hosted path at full utilisation. Azure fallback is orders of magnitude higher.

~3 + 1 vLLM calls

typical advisory flow — 3 MahaVistaar turns plus 1 moderation call. Varies by use case: ~2 on simple lookups, 4+ on complex flows.

100 concurrent cap

per vLLM endpoint. The 101<sup>st</sup> request spills to Azure OpenAI. The cap is tunable, not architectural.

---

---

## 01

### ANATOMY OF A QUESTION

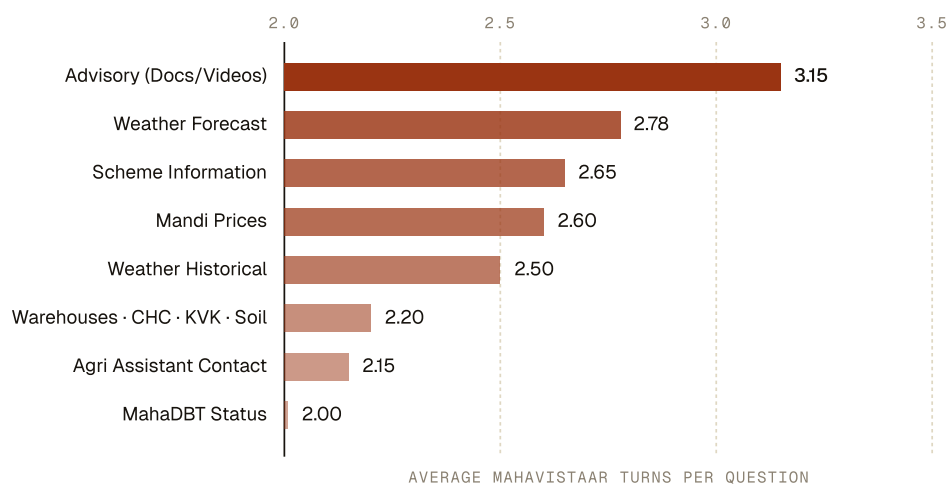
## What one advisory exchange actually costs

**W**hen a farmer asks MahaVistaar a question in Marathi, four things happen before the response begins to stream. The query passes through a moderation agent — GPT-OSS Safeguard 20B — which classifies it and gates unsafe or off-topic content. The conversational agent, our fine-tuned Qwen3.5-27B (**MahaVistaar LLM**), then decides which tools to invoke. It

processes the tool results — farm registry, weather, mandi prices, crop science, finance — and constructs a grounded, citeable answer. A background suggestions agent generates follow-up prompts while the user reads.

From the serving layer's perspective, that single exchange is typically **four vLLM calls**: one moderation call and three MahaVistaar calls. Moderation is tight — a 4,096-token context ceiling, ~2,500 tokens of instructions, ~100 tokens for the current question, optionally ~2,000 tokens of prior-turn context, and only 40–50 tokens of output. The call completes in **~200–300 ms** on the open-source model and is effectively free from a concurrency standpoint. The three MahaVistaar turns — tool selection, tool-result synthesis, and suggestions — hold a slot for ~5 seconds each on average and are where the system's capacity is spent.

**Three is a typical number, not a constant.** The turn count varies with query complexity. Advisory questions — our heaviest flow — average 3.15 MahaVistaar turns because they routinely chain multiple tool calls (weather + soil + crop-specific docs) before synthesis. Simpler lookups like MahaDBT status complete in two turns. Complex multi-step flows can run four or more when tool results require branching. The capacity model in Section 04 uses the ~3-turn average; the system itself doesn't care and scales with whatever shape the query takes.



**FIG. 01** Conversation complexity, production data. Advisory sits at the top of the turn distribution and is also two-thirds of total cost (see Section 06) — which is why architectural choices are tuned for this worst case.

The token budget per MahaVistaar turn is dominated by the conversation prefix rather than the user's query. A 6,000-token system prompt, a ~50-token user question, and accumulated tool outputs and prior turns regularly push the input side past 30,000 tokens by turn three. Without **prefix caching**, each turn would

re-pay the cost of the previous turn's prefix. With it, roughly 37% of notional input compute is skipped outright.

**Why the 84K context ceiling matters.** The max context window is set to 84,000 tokens — large enough to cover the longest realistic multi-turn advisory session with tool outputs, small enough that KV cache memory remains a per-request cost the GPU pool can absorb at the desired concurrency.

## 02

SERVING TOPOLOGY

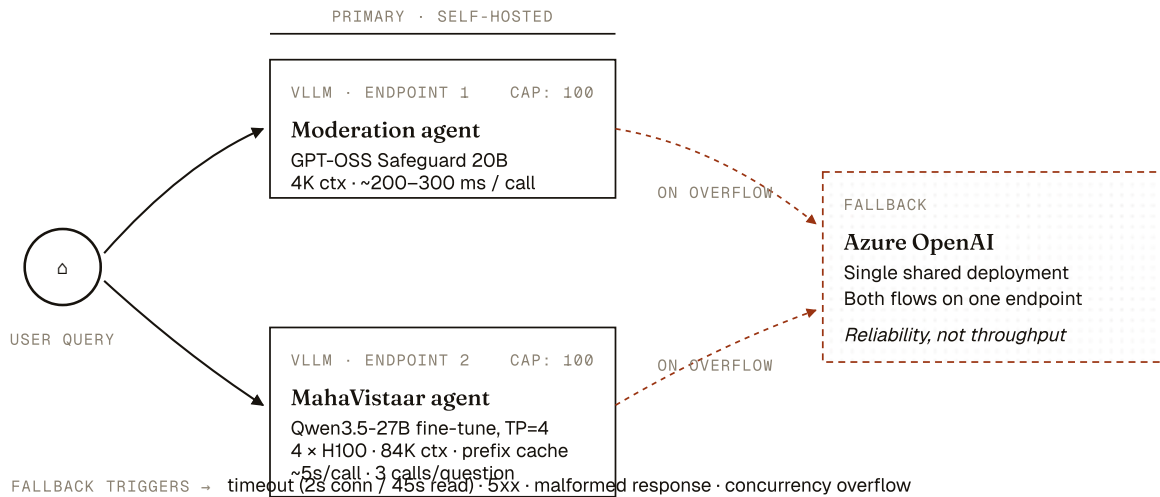
### Two primaries, one fallback, one policy

Every generative LLM call in the platform goes through a PydanticAI

`OpenAIChatModel` wrapper pointed at one of two providers. The active provider is resolved at import time from `LLM_PROVIDER`, and the stack is structured so that failover is automatic, deterministic, and transparent to the application.

- **Primary — vLLM, self-hosted.** Two independent endpoints: `VLLM_AGRINET_MODEL_URL` (MahaVistaar on 4 × H100, TP=4, 84K context, prefix caching on) and `VLLM_MODERATION_MODEL_URL` (GPT-OSS Safeguard 20B on 2 × H100). Failure of one endpoint does not affect the other.
- **Fallback — Azure OpenAI.** A single deployment shared by both flows. Its job is reliability, not steady-state throughput.

Failover triggers only on genuine infrastructure faults: connection timeouts (2 s connect, 45 s read), 5xx responses, malformed payloads, or concurrency overflow at the API limiter. It does *not* trigger on low-quality answers, moderation rejections, or 4xx client errors — those are application concerns and handled upstream.



**FIG. 02** Serving topology. Two independent vLLM endpoints on the primary path; a single Azure OpenAI deployment as shared fallback. Triggers are infrastructure-level only.

## 03 DESIGN POLICY

### Latency over queue depth

The vLLM primary is capped at 100 concurrent calls per endpoint. When a request arrives and the cap is saturated, the system does *not* queue it. The request is routed to Azure instantly, transparently, at the same API surface. vLLM continues serving its in-flight load at full speed; capacity reopens as requests complete.

“

*Users never experience elevated latency from queuing — even at burst load. The cost of that guarantee is paid by Azure, not by the user.*

— OPERATING POLICY, IN ONE LINE

Two things follow from this choice, and both matter.

**The 100-call cap is tunable, not architectural.** It lives in an environment variable, not in code. As Langfuse telemetry matures and we observe the real latency distribution at higher concurrency, we will raise the cap to push more steady-state traffic onto the cheaper self-hosted path. A small queue buffer can also be introduced if short bursts would benefit from brief waits rather than immediate fallback — today they do not.

**Blended cost per question is not a constant.** It depends on how often spillover fires. At the current cap, normal load sits well below 100 concurrent MahaVistaar calls, and spillover is a rare event — but we are inferring this, not measuring it. The next section sets out why, and the final section sets out what we plan to do about it.

---

## 04 CONCURRENT MODEL

### Little's Law, applied to a multi-turn advisory flow

From the serving layer's perspective, one user question is three MahaVistaar calls plus one moderation call, spread across a ~20-second user cycle (12–15 s wall-clock response, plus 5–8 s of read-and-type time). Each MahaVistaar call holds a concurrency slot for about 5 seconds on average.

Little's Law gives us the carrying capacity directly. Average slot occupancy per active user is the rate of calls times the time per call: 0.15 calls per second times 5 seconds, or **0.75 slots per user**. Divide the available slot budget by that number and you have the ceiling.

---

#### MAHAVISTAAR — CARRYING CAPACITY

$$\lambda_{\text{user}} = \frac{3 \text{ calls}}{20 \text{ s cycle}} = 0.15 \text{ calls/s}$$

$$\text{occupancy}_{\text{user}} = \lambda \cdot W = 0.15 \times 5 \text{ s} = \mathbf{0.75 \text{ slots/user}}$$

---

$$\text{capacity} = \frac{60 \text{ slots}}{0.75 \text{ slots/user}} \approx \mathbf{80} \text{ concurrent users}$$

*ideal, evenly-spaced traffic*

$$\text{safe operating range} = \mathbf{60\text{--}70} \text{ concurrent users}$$

*accounting for bursty arrival*

$$\text{moderation capacity} = \frac{100 \text{ slots}}{0.05 \times 0.25 \text{ s}} \approx \mathbf{8,000} \text{ users}$$

*at 200–300 ms per call, well off the critical path*

Empirical tests match the model. Load at 10, 25, and 60 users/min produced *no* fallback events: real users don't stagger their requests cleanly, but multi-turn calls are sequential within a question, so natural think-time smooths demand below the cap. The system behaves the way the math predicts.

**Where the "60 slots" comes from.** The API limiter is set to 100, matching vLLM's 128-request internal capacity with headroom. In practice we've modelled against 60 effective slots to absorb variance in per-call duration (some tool-result synthesis runs hot at 8 s) and to leave burst margin. As we raise the cap, this number moves with it.

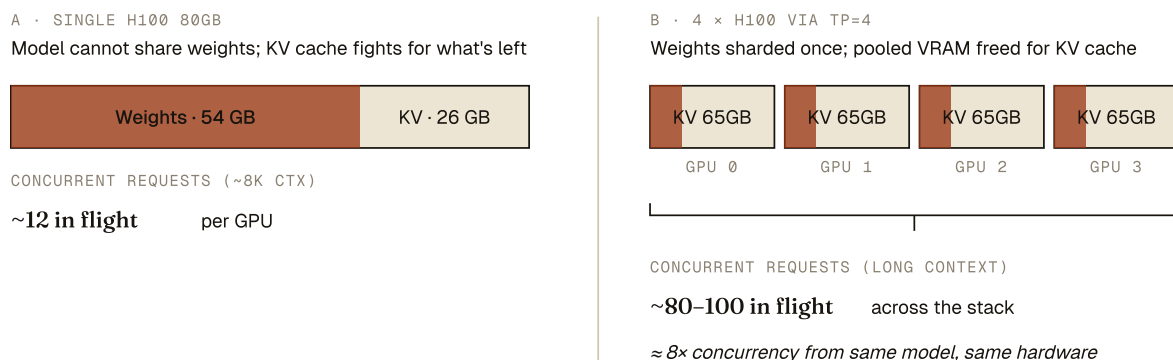
## 05 WHY TP=4

### Concurrency is bought with VRAM, not compute

The concurrency ceiling above is a memory story, not a compute story. A 27B-parameter model in FP16 occupies ~54GB of weights. On a single H100 80GB, that leaves ~26GB for KV cache — enough for roughly a dozen long-context requests in flight. The GPU's tensor cores would sit idle waiting for requests the VRAM couldn't hold.

Tensor parallelism at TP=4 shards the weights across four GPUs. Each GPU holds roughly a quarter of the model, and the remaining VRAM — ~65GB per GPU,

~260GB pooled — becomes KV cache. The same weights, served concurrently to an order of magnitude more requests.



**FIG. 03** Memory layout, single-GPU vs TP=4. Sharding weights once across four GPUs converts weight-replication overhead into usable KV cache — the currency of concurrency.

This is the architectural lesson: **for a model this size, concurrency — not latency — sets unit economics.** Tensor parallelism is the lever that turns four 80GB GPUs into a single 320GB pool. The same principle generalises — if we fine-tune something larger (40B, 70B), the TP decision is what makes or breaks the ₹/question number.

## 06

THE PREVIOUS BASELINE

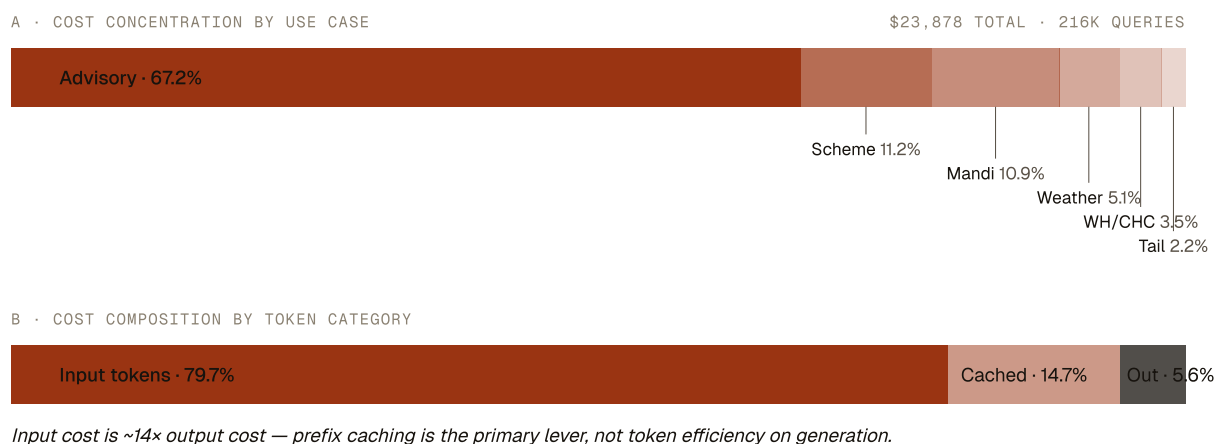
### What the platform used to cost, on fully-managed inference

The numbers in the previous sections describe the current self-hosted deployment. The baseline those numbers are being compared against is not hypothetical either: for several months before the migration, the platform ran entirely on GPT-4.1 via Azure OpenAI, and the bill is on the record.

November 2025 production data: **~₹2 lakh per day** in steady-state LLM spend, with the trajectory pointing toward **~₹6 lakh per day** as Voice AI went live and chatbot adoption scaled. Six months of GPU rental for the current 4-GPU cluster

cost ₹25 lakh; a planned 16-GPU build-out would absorb roughly ₹2 crore over the year — well under the projected Azure run-rate. The question wasn't whether to migrate to self-hosted. It was how aggressively, and under what reliability envelope.

Two patterns in the Azure spend shaped every architectural choice in the deployment that replaced it.



**FIG. 04** Azure OpenAI spend breakdown, November 2025. Top: Advisory dominates cost concentration. Bottom: input tokens dominate composition, with caching under-utilised on GPT-4.1 compared to what vLLM permits.

**Advisory was two-thirds of the bill.** It accounted for 67% of LLM spend on 57% of queries — heavier per question than any other flow. That concentration is why the serving stack is tuned to Advisory's worst case: three or more turns, long tool-result payloads, 84K context ceiling, TP=4 for concurrency. Making Advisory cheaper is making the platform cheaper.

**Input tokens were 80% of the bill.** Output was a rounding error at 5.6%; cached input was a narrow 14.7% wedge because prefix caching wasn't as aggressive on GPT-4.1 as vLLM allows. The implication is direct — on the self-hosted path, prefix caching isn't a micro-optimisation. It is the single biggest cost lever besides model choice itself. Section 01 noted that ~37% of notional input compute on a three-turn conversation is prior-turn prefix being re-sent; with caching, that compute is skipped.

The per-question picture, broken out:



| USE CASE                           | QUERIES         | AVG<br>TURNS | COST / Q<br>(₹) | COST / Q<br>(\$) |
|------------------------------------|-----------------|--------------|-----------------|------------------|
| Advisory (Docs/Videos)             | 1,23,319        | 3.15         | ₹11.06          | \$0.1301         |
| Scheme Information                 | 31,252          | 2.65         | ₹7.28           | \$0.0857         |
| Mandi Prices                       | 29,182          | 2.60         | ₹7.56           | \$0.0889         |
| Weather Forecast                   | 21,499          | 2.78         | ₹4.79           | \$0.0563         |
| Warehouses · CHC · KVK ·<br>Soil   | 5,409           | 2.20         | ₹13.09          | \$0.1540         |
| Agri Assistant Contact             | 2,529           | 2.15         | ₹8.74           | \$0.1028         |
| MahaDBT Status                     | 2,808           | 2.00         | ₹6.39           | \$0.0752         |
| Weather Historical                 | 418             | 2.50         | ₹8.98           | \$0.1056         |
| <b>Volume-weighted<br/>average</b> | <b>2,16,416</b> | <b>—</b>     | <b>₹9.4</b>     | <b>\$0.1103</b>  |

The volume-weighted average lands near ₹9 per question on Azure. Against the ₹0.05 per question cost on the self-hosted path — which the next section works through — that is a reduction on the order of **~180×**. Even accounting for less-than-full utilisation and spillover events, a two-order-of-magnitude move has been achieved. That gap is the architecture's reason for existing.

## 07 THE REPLACEMENT

### Unit economics on the self-hosted path

Against the ₹9-per-question baseline in Section 06, the self-hosted path's math is straightforward. At list H100 rental pricing of ~\$2/GPU/hour (~₹180), a 4-GPU MahaVistaar node costs ₹720/hour. At full utilisation — ~80 concurrent users

each producing ~3 questions/minute — the node serves ~240 questions/minute, or ~14,400/hour.

COST PER QUESTION · SELF-HOSTED PATH · LIST PRICE

$$\text{cost/Q} = \frac{\text{₹ 720/hour}}{14,400 \text{ Q/hour}} = \text{₹ 0.05 / question}$$

*4 × H100 at \$2/GPU/hour list rate, 80 concurrent users × 3 Q/min at full utilisation*

The actual rate is materially better. Six months of the current 4-GPU cluster has cost ₹25 lakh, working out to ~₹144/GPU/hour — roughly 20% below list. At that rate a 4-GPU node runs at ~₹575/hour and cost-per-question falls nearer ₹0.04. For this note's purposes we'll stay with the ₹0.05 figure as a conservative anchor; the real number is a little lower and can only improve as committed-use pricing deepens.

The monthly picture, using actual rental rates:

| CONFIGURATION                                        | RENTAL /<br>MONTH | QUESTIONS /<br>MONTH | MAU @ 4<br>Q/MO |
|------------------------------------------------------|-------------------|----------------------|-----------------|
| 1 × 4-GPU node (TP=4) — <i>current</i>               | ₹4.2 L            | ~10.4 M              | ~2.6 M          |
| 2 × 4-GPU nodes (HA /<br>redundancy)                 | ₹8.3 L            | ~20.8 M              | ~5.2 M          |
| 4 × 4-GPU nodes (16-GPU cluster)<br>— <i>planned</i> | ₹16.7 L           | ~41.6 M              | ~10.4 M         |

The 16-GPU build-out pencils at roughly **₹2 crore annually**. Against a projected Azure run-rate of ₹6 lakh/day (~₹18 crore/year) once Voice AI scales, that is a decisive cost delta — enough to absorb the operational overhead of running the cluster, the risk premium of self-hosted serving, and still leave the platform an order of magnitude cheaper to run.

**A caveat on the top-line number.** ₹0.05/question is the *marginal* cost at full utilisation on the self-hosted path. Real-world blended cost is higher, because (a)

utilisation is rarely at 100%, and (b) spillover to Azure OpenAI on bursty load is orders of magnitude more expensive per call. The Azure fallback path is a reliability lever, not a steady-state serving strategy. The design explicitly prefers paying the fallback premium over making users wait.

## 08

WHAT WE DON'T YET MEASURE

# Langfuse, and the three questions it will let us answer

Several of the numbers in this note are modelled, not observed. Until Langfuse is integrated — the server is provisioned; integration is planned for this week — we are inferring fallback frequency, blended cost, and real per-flow latency from production logs and load tests, not measuring them directly.

Once Langfuse is live, three things become visible:

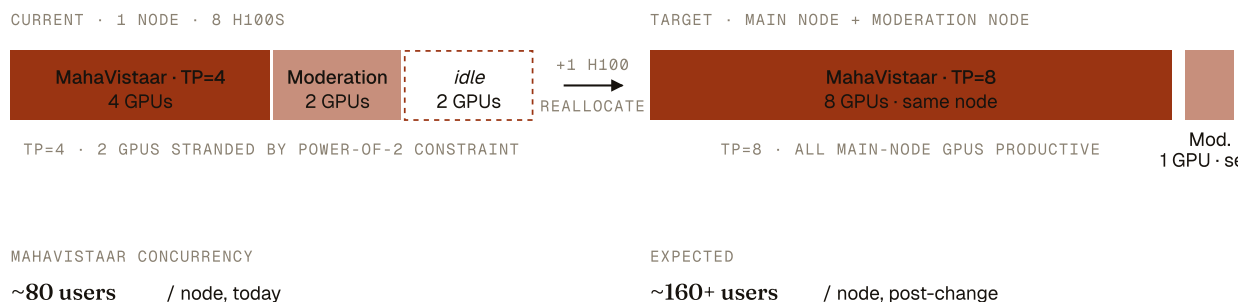
- **Provider split.** The fraction of traffic served by the self-hosted path versus Azure, over any window. This is the single most important operational number, because it determines the real blended cost per question and tells us whether the 100-call cap is set correctly.
- **Per-flow latency percentiles.** p50 / p95 / p99 for moderation, for each MahaVistaar turn, and for the full user-visible round-trip. This validates the "latency over queue depth" guarantee empirically rather than by assumption.
- **Fallback event correlation.** When does spillover fire? Under which traffic patterns? How correlated are events with GPU utilisation, tool-call latency, or particular user cohorts? This is the data we need to responsibly raise the cap.

The practical sequence from here is: land Langfuse → measure provider split for two weeks at current load → raise the concurrency cap in steps (120, 140) while watching p99 latency → formalise the safe-range SLA at whatever number the data supports. The 60–70-user safe range is today's SLA boundary; it is a starting point, not a ceiling.

## TP=8, and the arithmetic of moderation separation

The current deployment is a single node of eight H100s. Four of them run the main MahaVistaar LLM with TP=4. Two run the moderation agent — which really only needs one, given the ~200 ms call time and the 8,000-user capacity ceiling derived in Section 04. And two sit idle, because tensor parallelism on the main model accepts only power-of-two widths — 2, 4, or 8 — and the node can't step up to TP=8 without first freeing the GPUs held by moderation.

The fix is architectural, not financial. Spin up a separate single-H100 node dedicated to moderation. Consolidate the moderation allocation onto that one GPU, where it has ample headroom. Reclaim all eight GPUs on the main node for the MahaVistaar LLM at TP=8. A single added H100, one configuration change, and the main inference pool doubles in effective GPU count.



**FIG. 05** Current and target GPU allocation. Today's 8-GPU node strands two GPUs because TP can't step from 4 to 6. Adding a dedicated moderation H100 unlocks TP=8 and roughly doubles concurrency.

The throughput consequence is more than a clean doubling. The same principle from Section 05 applies — a larger TP shard leaves more pooled VRAM for KV cache, so concurrency scales faster than linearly with GPU count. At today's 3-questions-per-minute-per-user demand profile, the TP=8 configuration should serve 160+ concurrent users on the main node with headroom, and moderation stops competing for slots on the main pool entirely — the 100-concurrent cap applies to MahaVistaar calls alone.

The cost shape changes less than the capacity does. Cost per question on the main LLM stays close to the ₹0.05 figure from Section 07; the material move is that we stop paying rent on two idle GPUs, and the cluster's headroom to absorb the projected scale-up to ₹6 lakh/day of demand is unlocked without another round of capex. This is the main architectural lever left before the next model refresh.